

DBMS – PAYMENT MANAGEMENT SYSTEM

Project Title: IKEA E-Commerce Database Management System

1. Database Design

1.1 Payment Table

Attribute	Data Type	Key	Description
Payment_ID	INT	PK	Unique payment ID
Order_ID	INT	FK	References the order
Customer_ID	INT	FK	Customer who made the payment
Payment_Mode	VARCHAR(30)	-	Mode used for payment
Payment_Date	DATE	-	Date of payment
Payment_Amount	DECIMAL(10,2)	-	Amount paid
Payment_Status	VARCHAR(30)	-	Successful or Failed
Transaction_Reference	VARCHAR(50)	-	Unique transaction reference

Payment Modes

- UPI
- Credit Card
- Debit Card
- Net Banking
- Cash on Delivery

Payment Status

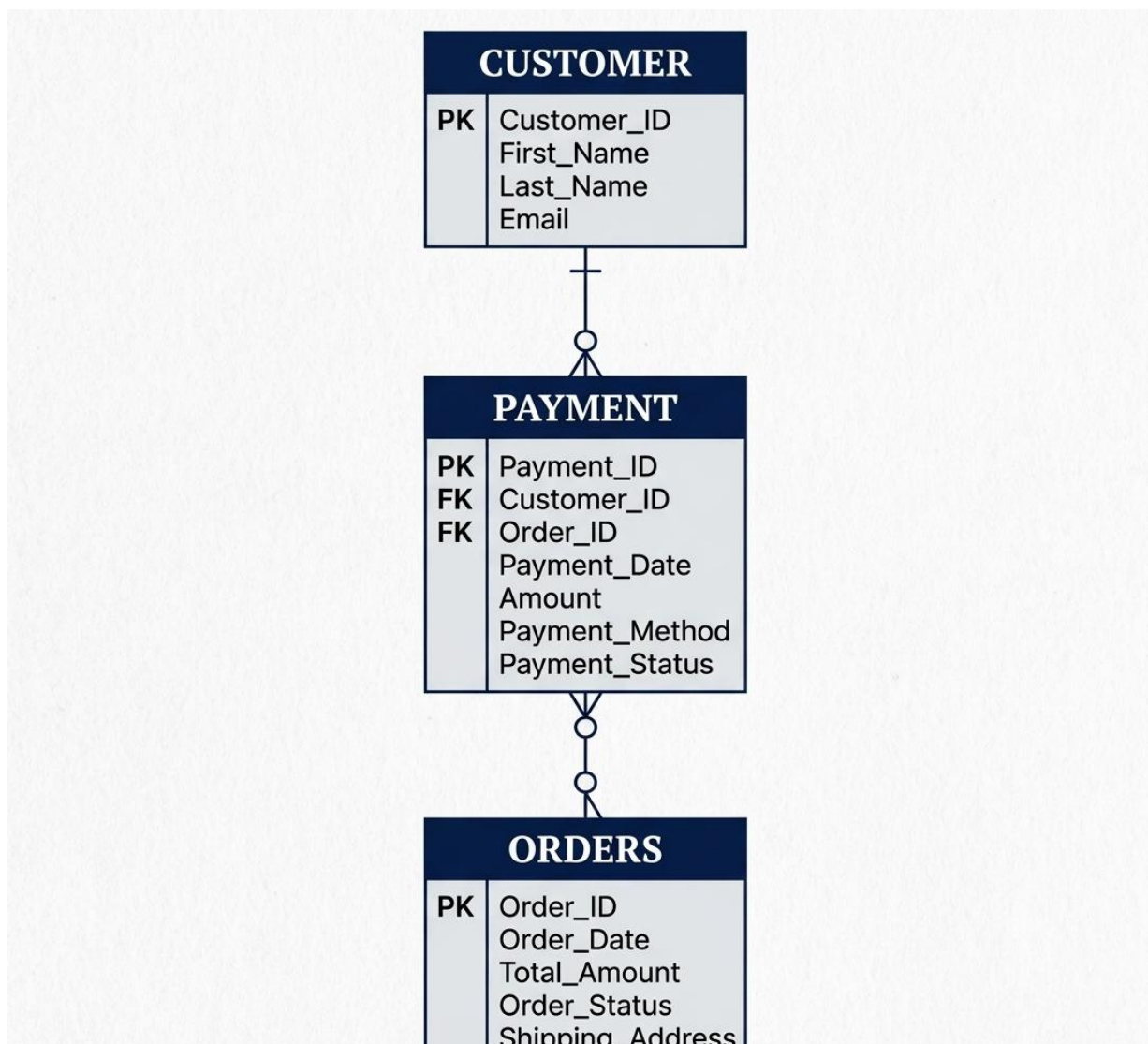
Use:

- Successful
- Failed

This directly covers the task requirements of storing the payment mode, date, status, and managing successful/failed transactions.

2. Table Relationship

A customer can make multiple payments, while each payment is associated with one customer. An order can have a payment transaction, and each payment is associated with an order.



3. Table Creation

3.1 Create Payment Table

```
SQL> CREATE TABLE Payment (  
2     Payment_ID INT PRIMARY KEY,  
3     Order_ID INT,  
4     Customer_ID INT,  
5     Payment_Mode VARCHAR2(30),  
6     Payment_Date DATE,  
7     Payment_Amount DECIMAL(10,2),  
8     Payment_Status VARCHAR2(30),  
9     Transaction_Reference VARCHAR2(50)  
10 );
```

Table created.

4. Payment Data Insertion

```
SQL> INSERT INTO Payment VALUES  
2 (1, 1001, 1, 'UPI', DATE '2026-09-01', 12999.00, 'Successful', 'TXN100001');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (2, 1002, 2, 'Credit Card', DATE '2026-09-03', 6999.00, 'Successful', 'TXN100002');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (3, 1003, 3, 'Debit Card', DATE '2026-09-05', 5999.00, 'Failed', 'TXN100003');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (4, 1004, 4, 'Net Banking', DATE '2026-09-07', 4999.00, 'Successful', 'TXN100004');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (5, 1005, 5, 'UPI', DATE '2026-09-10', 3999.00, 'Successful', 'TXN100005');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (6, 1006, 6, 'Credit Card', DATE '2026-09-12', 1499.00, 'Failed', 'TXN100006');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (7, 1007, 7, 'Debit Card', DATE '2026-09-15', 1999.00, 'Successful', 'TXN100007');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (8, 1008, 8, 'Cash on Delivery', DATE '2026-09-18', 8999.00, 'Successful', 'TXN100008');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (9, 1009, 9, 'UPI', DATE '2026-09-20', 2499.00, 'Failed', 'TXN100009');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Payment VALUES  
2 (10, 1010, 10, 'Net Banking', DATE '2026-09-22', 799.00, 'Successful', 'TXN100010');  
  
1 row created.  
  
SQL>  
SQL> COMMIT;
```

5. Retrieve Payment Details

The Payment table was retrieved using the SELECT statement to verify the payment details stored in the database.

```
SQL> SELECT * FROM Payment;
```

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_MODE	PAYMENT_D

PAYMENT_AMOUNT PAYMENT_STATUS				

TRANSACTION_REFERENCE				

	1	1001	1 UPI	01-SEP-26
	12999	Successful		
TXN100001				
	2	1002	2 Credit Card	03-SEP-26
	6999	Successful		
TXN100002				
PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_MODE	PAYMENT_D

PAYMENT_AMOUNT PAYMENT_STATUS				

TRANSACTION_REFERENCE				

	3	1003	3 Debit Card	05-SEP-26
	5999	Failed		
TXN100003				
	4	1004	4 Net Banking	07-SEP-26
	4999	Successful		
PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_MODE	PAYMENT_D

PAYMENT_AMOUNT PAYMENT_STATUS				

TRANSACTION_REFERENCE				

TXN100004				
	5	1005	5 UPI	10-SEP-26
	3999	Successful		
TXN100005				
	6	1006	6 Credit Card	12-SEP-26
PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_MODE	PAYMENT_D

PAYMENT_AMOUNT PAYMENT_STATUS				

TRANSACTION_REFERENCE				

TRANSACTION_REFERENCE				

		1499	Failed	
TXN100006				
		7	1007	7 Debit Card
		1999	Successful	15-SEP-26
TXN100007				
PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_MODE	PAYMENT_D

PAYMENT_AMOUNT	PAYMENT_STATUS			

TRANSACTION_REFERENCE				

		8	1008	8 Cash on Delivery
		8999	Successful	18-SEP-26
TXN100008				
		9	1009	9 UPI
		2499	Failed	20-SEP-26
TXN100009				
PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_MODE	PAYMENT_D

PAYMENT_AMOUNT	PAYMENT_STATUS			

TRANSACTION_REFERENCE				

		10	1010	10 Net Banking
		799	Successful	22-SEP-26
TXN100010				
10 rows selected.				

6. Successful Transactions

Successful payment transactions were identified by filtering the Payment table using the payment status. This helps verify the transactions that were completed successfully.

```

SQL> SELECT
  2   Payment_ID,
  3   Order_ID,
  4   Customer_ID,
  5   Payment_Mode,
  6   Payment_Amount,
  7   Payment_Status
  8 FROM Payment
  9 WHERE Payment_Status = 'Successful';

```

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_MODE	PAYMENT_AMOUNT
1	1001		1 UPI	12999
2	1002		2 Credit Card	6999
4	1004		4 Net Banking	4999
5	1005		5 UPI	3999
7	1007		7 Debit Card	1999
8	1008		8 Cash on Delivery	8999
10	1010		10 Net Banking	799

7 rows selected.

7. Failed Transactions

Failed payment transactions were retrieved using the Payment_Status condition. These records help identify transactions that require attention or possible retry.

```
SQL> SELECT
  2     Payment_ID,
  3     Order_ID,
  4     Customer_ID,
  5     Payment_Mode,
  6     Payment_Amount,
  7     Payment_Status
  8 FROM Payment
  9 WHERE Payment_Status = 'Failed';
```

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_MODE	PAYMENT_AMOUNT
Failed	3	1003	3 Debit Card	5999
Failed	6	1006	6 Credit Card	1499
Failed	9	1009	9 UPI	2499

3 rows selected.

8. Analyze Payment Methods Used by Customers

Payment methods used by customers were analyzed using the COUNT and SUM aggregate functions. The report displays the number of successful transactions and the total payment amount for each payment mode.

```
SQL> SELECT
  2     Payment_Mode,
  3     COUNT(*) AS Number_of_Transactions,
  4     SUM(Payment_Amount) AS Total_Amount
  5 FROM Payment
  6 WHERE Payment_Status = 'Successful'
  7 GROUP BY Payment_Mode
  8 ORDER BY Number_of_Transactions DESC;
```

PAYMENT_MODE	NUMBER_OF_TRANSACTIONS	TOTAL_AMOUNT
UPI	2	16998
Net Banking	2	5798
Cash on Delivery	1	8999
Debit Card	1	1999
Credit Card	1	6999

9. Payment Transaction Report

A payment transaction report was generated by combining payment and order information. The report displays the order, payment mode, payment date, amount, status, and transaction reference.

```
SQL> SELECT
  2     p.Payment_ID,
  3     p.Order_ID,
  4     p.Customer_ID,
  5     o.Order_Date,
  6     p.Payment_Mode,
  7     p.Payment_Date,
  8     p.Payment_Amount,
  9     p.Payment_Status,
10     p.Transaction_Reference
11 FROM Payment p
12 JOIN Orders o
13 ON p.Order_ID = o.Order_ID
14 ORDER BY p.Payment_ID;
```

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	ORDER_DATE	PAYMENT_MODE

PAYMENT_ID	PAYMENT_AMOUNT	PAYMENT_STATUS		

TRANSACTION_REFERENCE				

	1	1001	1 02-SEP-26	UPI
01-SEP-26	12999	Successful		
TXN100001				

	2	1002	2 03-SEP-26	Credit Card
03-SEP-26	6999	Successful		
TXN100002				

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	ORDER_DATE	PAYMENT_MODE

PAYMENT_ID	PAYMENT_AMOUNT	PAYMENT_STATUS		

TRANSACTION_REFERENCE				

	3	1003	3 05-SEP-26	Debit Card
05-SEP-26	5999	Failed		
TXN100003				

	4	1004	4 07-SEP-26	Net Banking
07-SEP-26	4999	Successful		

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	ORDER_DATE	PAYMENT_MODE

PAYMENT_ID	PAYMENT_AMOUNT	PAYMENT_STATUS		

TRANSACTION_REFERENCE				

TXN100004				

	5	1005	5 10-SEP-26	UPI
10-SEP-26	3999	Successful		
TXN100005				

	6	1006	6 12-SEP-26	Credit Card
--	---	------	-------------	-------------

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	ORDER_DATE	PAYMENT_MODE

PAYMENT_ID	PAYMENT_AMOUNT	PAYMENT_STATUS		

```

| 5 | 1005 | 5 | 10-SEP-26 | UPI
10-SEP-26 | 3999 | Successful
TXN100005

| 6 | 1006 | 6 | 12-SEP-26 | Credit Card

PAYMENT_ID ORDER_ID CUSTOMER_ID ORDER_DAT PAYMENT_MODE
-----
PAYMENT_D PAYMENT_AMOUNT PAYMENT_STATUS
-----
TRANSACTION_REFERENCE
-----
12-SEP-26 | 1499 | Failed
TXN100006

| 7 | 1007 | 7 | 15-SEP-26 | Debit Card
15-SEP-26 | 1999 | Successful
TXN100007

PAYMENT_ID ORDER_ID CUSTOMER_ID ORDER_DAT PAYMENT_MODE
-----
PAYMENT_D PAYMENT_AMOUNT PAYMENT_STATUS
-----
TRANSACTION_REFERENCE
-----

| 8 | 1008 | 8 | 18-SEP-26 | Cash on Delivery
18-SEP-26 | 8999 | Successful
TXN100008

| 9 | 1009 | 9 | 20-SEP-26 | UPI
20-SEP-26 | 2499 | Failed
TXN100009

PAYMENT_ID ORDER_ID CUSTOMER_ID ORDER_DAT PAYMENT_MODE
-----
PAYMENT_D PAYMENT_AMOUNT PAYMENT_STATUS
-----
TRANSACTION_REFERENCE
-----

| 10 | 1010 | 10 | 22-SEP-26 | Net Banking
22-SEP-26 | 799 | Successful
TXN100010

10 rows selected.

```

10. Payment Status Summary

GROUP BY Payment_Status;

This will show the number and total amount of:

- Successful transactions
- Failed transactions

Payment transactions were grouped according to their status using the COUNT and SUM aggregate functions. The report provides an overview of successful and failed payment transactions.

```
SQL> SELECT
  2     Payment_Status,
  3     COUNT(*) AS Transaction_Count,
  4     SUM(Payment_Amount) AS Total_Amount
  5 FROM Payment
  6 GROUP BY Payment_Status;
```

PAYMENT_STATUS	TRANSACTION_COUNT	TOTAL_AMOUNT
Successful	7	48793
Failed	3	9997